

Unit-3 CONFIGURATION MANAGEMENT TOOLS

Prof. Prachi Jain

CONTENT

- ▶ Introduction to Configuration Management tools
- ▶ Types of Configuration Management Tools: Push-based & Pull-based
- ▶ Introduction to Ansible, Puppet, Chef & Salt
- ▶ Ansible : What is Ansible and its Architecture
- ▶ Why do we need Ansible, Ansible Terminologies, Advantages
- ▶ Infrastructure-as-a-code
- ▶ Writing Ansible Playbooks using YAML

Introduction to Configuration Management

- ▶ **Configuration Management (CM)** is a systems engineering process that allows you to *configure your infrastructure and automate many tasks.*
- ▶ Configuration Management (CM) is the process of systematically
 - ▶ Managing
 - ▶ Organizing
 - ▶ Maintaining
 - ▶ the configuration of computer systems, servers, applications, and network devices throughout their lifecycle.
- ▶ It ensures that systems remain consistent, reliable, and secure by automating the deployment, configuration, monitoring, and updating of infrastructure.
- ▶ In DevOps, Configuration Management plays a crucial role by eliminating manual configuration errors and ensuring that all environments (development, testing, and production) are configured identically.

- ▶ Configuration Management is the practice of maintaining the desired state of IT infrastructure using automation tools.
- ▶ It tracks changes, enforces standard configurations, and ensures that systems remain compliant with organizational policies.
- ▶ In the last decade, a *plethora of configuration management tools hit the software space.
- ▶ Tools like Ansible, Puppet, Salt, Chef, Terraform and many more offer a wide range of capabilities to ease your infrastructure development.
- ▶ While they have many overlapping features, each software offers a unique set of tools.

*Plethora - large amount of

Why is Configuration Management important?

Before configuration management, system administrators configured servers manually, leading to several problems:

- ▶ Configuration inconsistencies between servers
- ▶ Human errors during deployment
- ▶ Difficult troubleshooting
- ▶ Time-consuming setup process
- ▶ Poor scalability
- ▶ Security vulnerabilities due to misconfigurations

Configuration Management tools solve these problems through automation.

- ▶ In the era of microservices, we shouldn't configure each service manually: it's time-consuming and prone to human error.
- ▶ **Configuration management allows you to centralize and automate your infrastructure configuration.**
- ▶ An average web application has at least one database, a shared cache server, and a few web app instances.
- ▶ When scaling horizontally, it's critical to be able to change configuration quickly and revert in the case of disaster.
- ▶ To minimize user impact, the architecture should be resilient and immune to any outages.
- ▶ CM enables you to tame your infrastructure, reduce operation costs and cut downtime.
- ▶ Think of configuration management as an architectural framework of a house - it provides a strong base for streamlined development.

*Tame - under control

Objectives of Configuration Management

- Maintain consistency across systems
- Automate server provisioning and configuration
- Reduce manual intervention
- Improve system reliability
- Simplify software deployment
- Track configuration changes
- Ensure compliance with security standards
- Support continuous integration and continuous deployment (CI/CD)

Features of Configuration Management Tools

- Automated software installation
- Infrastructure as Code (IaC)
- Configuration enforcement
- Version control integration
- Remote server management
- Scalability
- Agent-based or agentless architecture
- Reporting and auditing
- Rollback capability
- Idempotent operations (same result even if executed multiple times)

Types of Configuration Management Tools

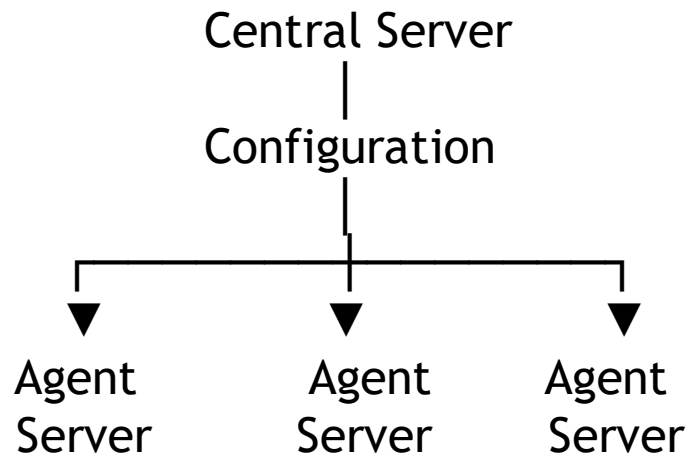
- ▶ Configuration Management (CM) tools can be broadly classified based on **how they manage configurations and communicate with target systems.**
- ▶ Based on Architecture
 - ▶ Agent-Based
 - ▶ Agentless
- ▶ Based on Execution Model
 - ▶ Push Model
 - ▶ Pull Model
- ▶ Based on Language & Methodology
 - ▶ Declarative (Desired State)
 - ▶ Imperative / Procedural

1. Server-Based / Agent-Based Tools

- ▶ These tools require a **client/agent software** to be installed on the target server.
- ▶ The agent communicates with a central management server and applies the required configuration.

▶ Examples:

- ▶ Puppet
- ▶ Chef
- ▶ CFEngine



Advantages:

- Continuous configuration monitoring
- Automatic correction of configuration changes
- Suitable for large infrastructures

Disadvantages:

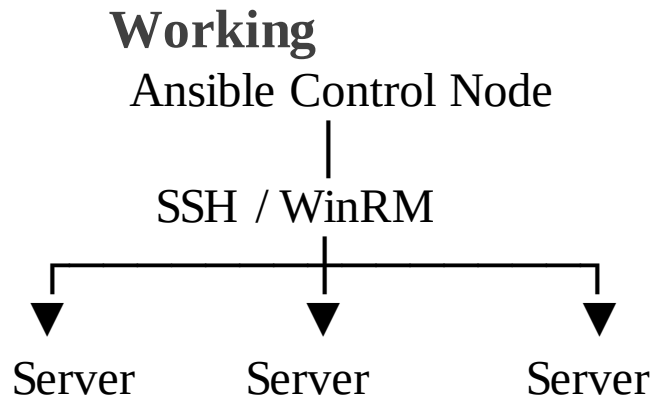
- Agent installation and maintenance are required
- More complex initial setup

2. Agentless Configuration Management Tools

- ▶ Agentless tools **do not require special software to be installed on target machines.**
- ▶ They generally use existing communication protocols such as SSH or WinRM.
- ▶ **SSH** stands for **Secure Shell**. It is mainly used to securely connect to **Linux/Unix systems**.
- ▶ **WinRM** stands for **Windows Remote Management**. It is mainly used to remotely manage **Windows systems**.

- ▶ **Example:**

- ▶ **Ansible**



Advantages:

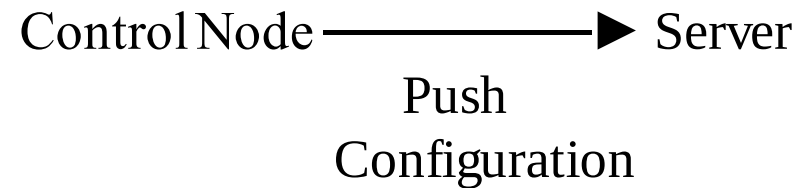
- No agent installation
- Easy to configure
- Simple architecture
- Easy to start with

Disadvantages:

- Requires network connectivity
- Continuous monitoring may require additional mechanisms

3. Push-Based Configuration Management

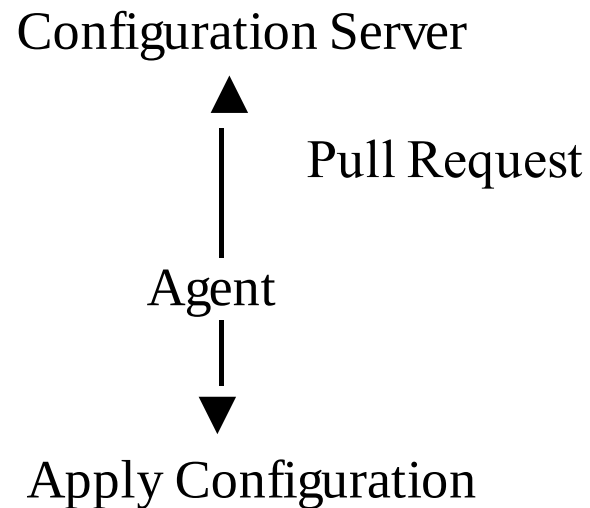
- ▶ In a **push model**, the administrator or control server actively sends configuration changes to target machines.
- ▶ **Example:**
 - ▶ Ansible



For example, an administrator can execute an Ansible playbook and push the required package, user, or service configuration to multiple servers.

4. Pull-Based Configuration Management

- ▶ In a **pull model**, the target system periodically contacts the configuration server and **pulls its desired configuration**.
- ▶ **Examples:**
 - ▶ Puppet
 - ▶ Chef
 - ▶ CFEngine



This approach is useful when systems need to continuously maintain their desired configuration.

5. Declarative Configuration Management Tools

- ▶ Declarative tools allow administrators to specify **what the desired state should be**, rather than describing every step required to achieve it.
- ▶ **Examples:**
 - ▶ Ansible
 - ▶ Puppet
 - ▶ Chef
- ▶ Example:
`web_server: state: installed`

The tool determines how to achieve the desired state.

6. Imperative Configuration Management Tools

- ▶ Imperative approaches specify **how the configuration should be achieved**, using a sequence of commands or instructions.
- ▶ For example:
 1. Install Apache
 2. Start Apache service
 3. Copy configuration file
 4. Restart Apache

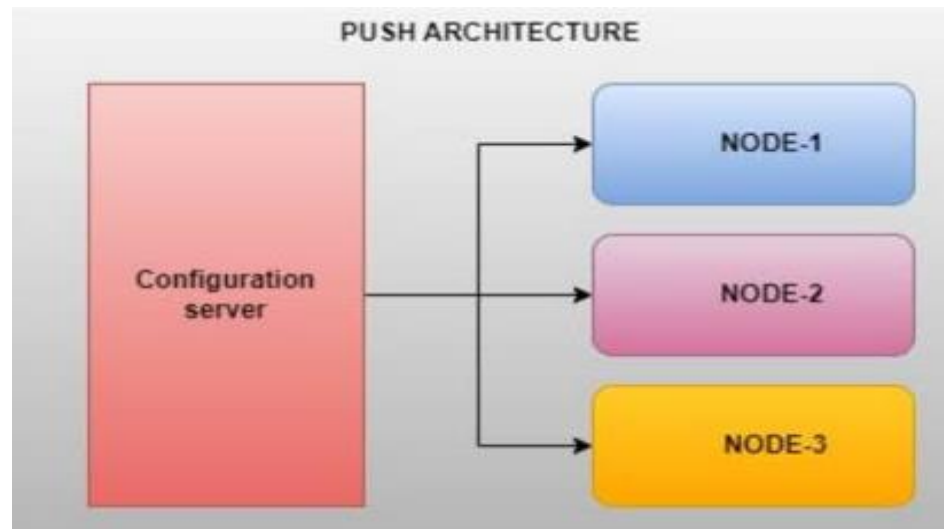
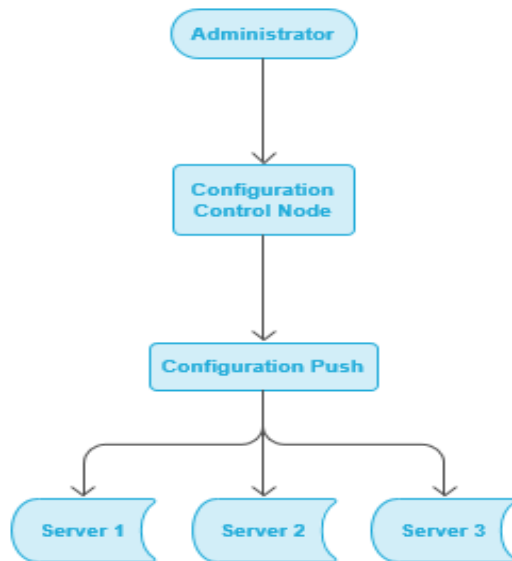
The administrator explicitly defines the operations to be performed.

Quick review/ Classification

Type	Description	Examples
Agent-Based	Requires agent on target system	Puppet, Chef, CFEngine
Agentless	No dedicated agent required	Ansible
Push-Based	Configuration is pushed to servers	Ansible, SaltStack
Pull-Based	Servers pull configuration periodically	Puppet, Chef
Declarative	Defines the desired state	Ansible, Puppet, Chef
Imperative	Defines the sequence of actions	Scripts/command-based tools

Push-Based Configuration Management

- ▶ In a **push-based model**, a central control machine or administrator **initiates the configuration process and pushes configuration instructions to the target servers**.
- ▶ The target server does not normally need to continuously request configuration updates. Whenever the administrator wants to make a change, the control node sends the required configuration to the target systems.
- ▶ Basic Architecture



How Push-Based Configuration Works

- ▶ Suppose an organization has 100 Linux servers and wants to install **Apache Web Server** on all of them.
- ▶ The administrator creates a configuration or playbook specifying:
 - ▶ Install Apache
 - ▶ Start Apache
 - ▶ Enable Apache at boot
- ▶ The control node then connects to the target servers and sends the required commands/configuration.

► Steps

1. Administrator defines the desired configuration.
2. Configuration is stored in a playbook, script, or configuration file.
3. Administrator starts the configuration process.
4. Control node connects to target servers.
5. Configuration is pushed to the selected servers.
6. Target servers apply the configuration.
7. The tool reports the execution result.

► Example: Ansible

- Ansible is a commonly used example of a push-based and agentless configuration management tool.
- The administrator can execute an Ansible playbook from the control node, and Ansible pushes the required configuration to the selected machines.

► **Advantages of Push-Based Configuration**

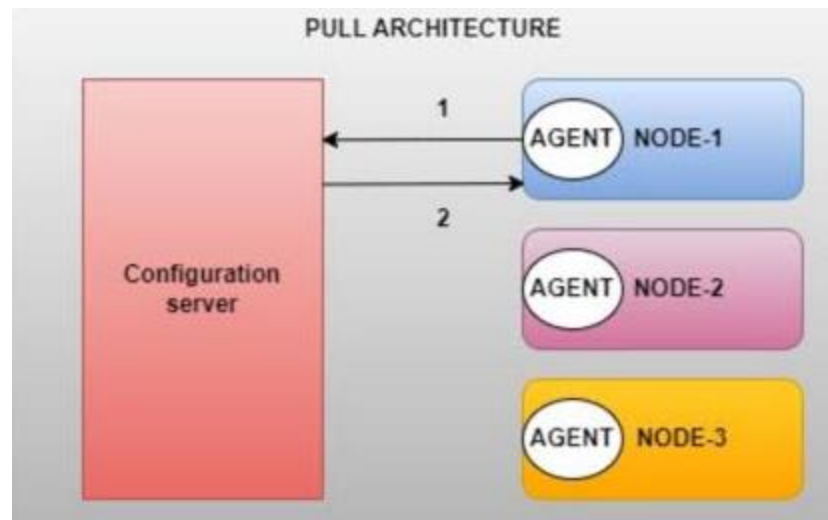
- **Simple architecture:** Usually only the control node needs the configuration-management software.
- **Fast deployment:** Changes can be applied immediately when the administrator initiates them.
- **Centralized control:** Administrators have direct control over when and where changes are applied.
- **Easy for on-demand changes:** Useful when a particular configuration needs to be deployed immediately.
- **Suitable for CI/CD:** Configuration changes can be integrated into automated deployment pipelines.

► **Disadvantages**

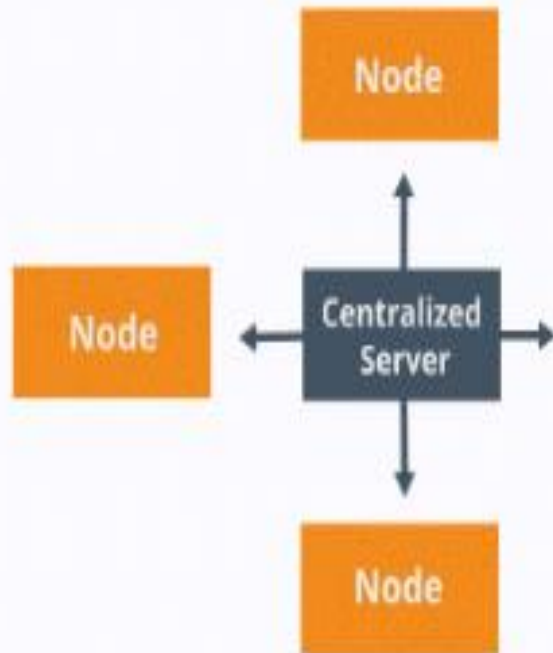
- The control node generally needs connectivity to the target machines.
- Large-scale deployments can place a significant load on the control node.
- If a server is unavailable when the push occurs, the configuration may not be applied unless another execution is scheduled.
- Requires appropriate authentication and network access.

2. Pull-Based Configuration Management

- In a **pull-based model**, the target machines **periodically contact a central configuration server and pull their configuration**.
- Instead of the administrator directly sending configuration to every server, an agent installed on the target system checks the configuration server and determines whether changes are required.
- Basic Architecture

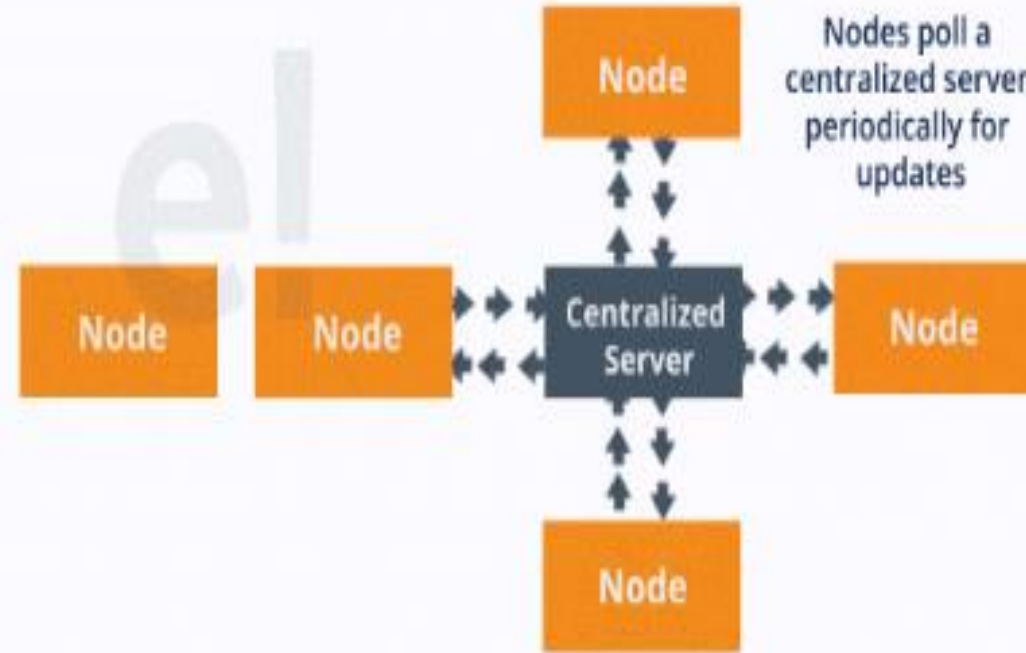


Push Configuration



Centralized sever pushes configurations on the nodes

Pull Configuration



Nodes dynamically update themselves with the configurations present in the server

How Pull-Based Configuration Works

- ▶ Suppose an organization manages 100 servers using a configuration management server.
- ▶ Each server has a configuration-management agent installed.
- ▶ The agent periodically contacts the configuration server:
 - ▶ "Do you have any configuration changes for me?"
- ▶ The server provides the required configuration, and the agent applies it.

► Steps

- Administrator defines the desired configuration.
- Configuration is stored on the central configuration server.
- An agent runs on each target machine.
- The agent periodically contacts the configuration server.
- It checks the desired configuration.
- If changes are available, the agent downloads them.
- The agent applies the configuration locally.
- The system reports the status back to the configuration server.

► Common examples include:

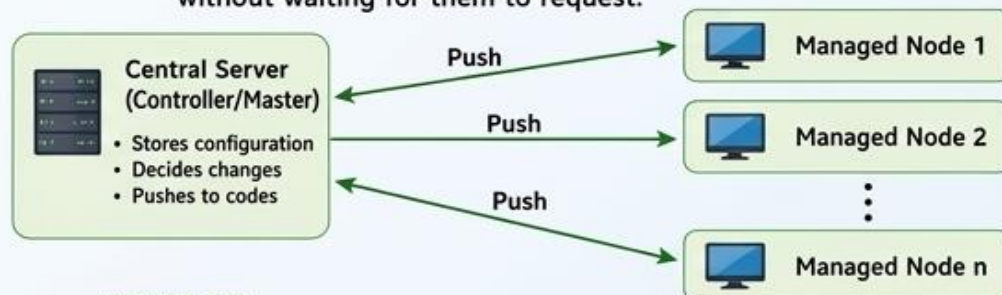
- **Puppet**
- **CFEngine**
- **Chef** in its traditional client-server model

PUSH AND PULL BASED CONFIGURATION MANAGEMENT

Two Approaches to Distribute and Apply Configurations to Managed Nodes

PUSH BASED CONFIGURATION MANAGEMENT

In push model, the central server (controller/master) sends the configuration to the managed nodes without waiting for them to request.



WORKFLOW

- 1 Administrator updates the configuration on the central server.
- 2 Server detects the change.
- 3 Server pushes the configuration to all (or selected) managed nodes.
- 4 Nodes receive the configuration and apply it.
- 5 Server may get feedback/status from nodes.

EXAMPLE TOOLS

- Ansible (Agentless, SSH-based push model)
- SaltStack (Master-Minion high-speed message bus push)

ADVANTAGES

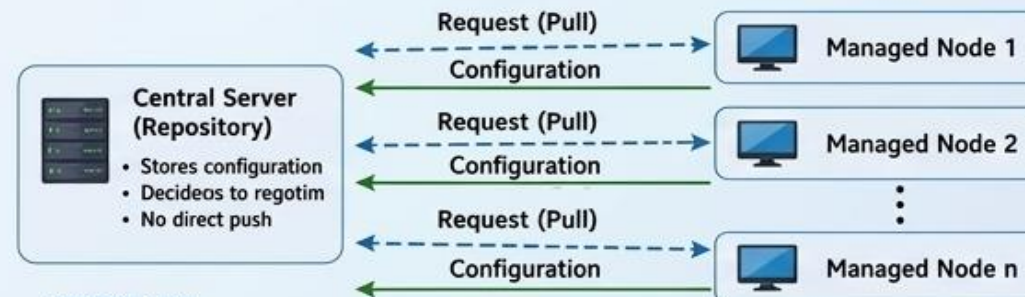
- Immediate delivery of changes
- Good for critical or urgent changes
- Centralized control

DISADVANTAGES

- Requires constant or constant from server to nodes
- Scalability issues for large infrastructure
- Higher load on server

PULL BASED CONFIGURATION MANAGEMENT

In pull model, managed nodes periodically request (pull) the configuration from the central server.



WORKFLOW

- 1 Node starts and requests configuration from the server.
- 2 Server sends the configuration to the node.
- 3 Node applies the configuration.
- 4 Node periodically repeats the process (at fixed intervals).
- 5 Node may report status or compliance to the server.

EXAMPLE TOOLS

- Puppet (Agent-driven polling for state convergence)
- Chef (Chef-client executes periodic convergence runs)
- CFEngine / Kubernetes (Controller-loop desired)

ADVANTAGES

- Scales well for large environments
- Works even behind firewalls / NAT
- Less load on the central server

DISADVANTAGES

- Changes are not immediate (depends on pull interval)
- Slower in applying critical updates
- Nodes must be configured to poll the server

PUSH vs PULL — AT A GLANCE

Aspect	Push vs Pull	Pull Based
Initiation	Initiated by central server	Initiated by managed node
Communication	Server → Node	Node → Server → Node
Change Delivery	Immediate	After next pull interval
Scalability	Limited for large scale	High scalability
Server Load	High	Low
Use Case	Small/medium environments, urgent changes	Large environments, secure networks



NOTE: This graphic reflects the primary architected workflow of each tool. Puppet and Chef are fundamentally pull-based with agents. SaltStack is fundamentally a high-speed push system. Manual kick commands exist for pull tools, but the primary method is agent polling. Similarly, Salt can be configured to operate differently, but push is its core design.



GOAL

Ensure the desired configuration (state) is consistently applied to all managed nodes in the most efficient and reliable way.

Push-Based vs Pull-Based Configuration Management

Feature	Push-Based	Pull-Based
Initiator	Control server/administrator	Target server/agent
Configuration flow	Control node → Target	Target → Configuration server
Agent	Often not required	Usually required
Example	Ansible, SaltStack	Puppet, CFEngine
Execution	On demand	Periodically/automatically
Control	Centralized	Distributed

Push-Based vs Pull-Based Configuration Management

Feature	Push-Based	Pull-Based
Connectivity	Control node needs access to targets	Agents need access to configuration server
Immediate deployment	Easy	Usually depends on agent polling interval
Offline server	May miss the push	Can pull configuration when it reconnects
Architecture	Relatively simple	More infrastructure required
Scalability	Good, but control node manages pushes	Very good for continuous management
Configuration enforcement	Usually when push is executed	Continuous/periodic enforcement
Typical use	Deployment and on-demand changes	Continuous configuration maintenance

Introduction to Ansible, Puppet, Chef & Salt

- ▶ Ansible: An agentless automation tool that uses YAML playbooks to manage multiple servers through SSH.
- ▶ Puppet: A configuration management tool that uses a master-agent architecture to maintain systems in a desired state.
- ▶ Chef: An automation tool that uses Ruby-based recipes and cookbooks to configure servers.
- ▶ SaltStack: A high-speed configuration management and remote execution tool that uses a master-minion architecture to manage large numbers of servers.

Tool	Think of it as...
Ansible	One remote control for many computers [One Remote → Many Computers]
Puppet	A strict teacher who keeps every computer following the rules [Teacher checks everyone]
Chef	A cook using recipes and cookbooks to prepare servers A Recipe configures one task. A Cookbook sets up an entire server. [Recipes + Cookbooks = Server Setup]
SaltStack	A fast messenger sending commands to thousands of computers instantly [Fast Messenger for Thousands of Servers.]

Ansible



ANSIBLE

- **What is it?** An agentless automation tool that manages multiple servers.
- **How it works?** Uses SSH to connect to servers. No agent installation required.
- **Language:** YAML (Playbooks)
- **Best for:** Automation, software installation, cloud deployment, configuration.
- **Advantage:** Easy to learn, lightweight, fast deployment.

Architecture (Agentless)



Example Playbook (YAML)

```
- hosts: all
  tasks:
    - name: Install Apache
      apt:
        name: apache2
        state: present
```

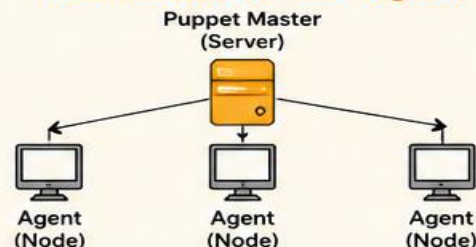
Puppet



puppet

- **What is it?** Configuration management tool that keeps systems in the desired state.
- **How it works?** Uses Master-Agent model. Agent is installed on every node.
- **Language:** Puppet DSL
- **Best for:** System configuration, compliance, large enterprises.
- **Advantage:** Automatically fixes changes, maintains consistency.

Architecture (Master-Agent)



Example Manifest (Puppet DSL)

```
package { 'apache2':
  ensure => installed,
}
```

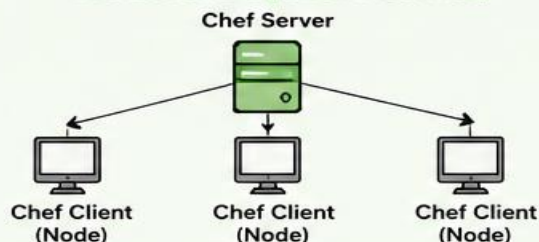
Chef



CHEF

- **What is it?** Automation tool that uses recipes and cookbooks to configure servers.
- **How it works?** Uses Chef Server and Chef Client (agent) for communication.
- **Language:** Ruby
- **Best for:** Complex infrastructures, reusable configurations.
- **Advantage:** Powerful, flexible, reusable cookbooks.

Architecture (Client-Server)



Example Recipe (Ruby)

```
package 'apache2' do
  action :install
end
```

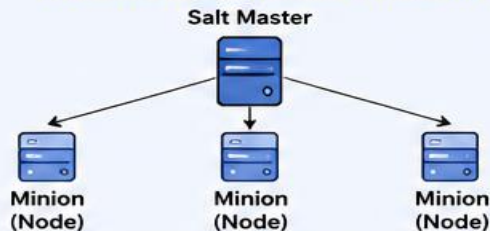
SaltStack (Salt)



SALTSTACK

- **What is it?** High-speed configuration management and remote execution tool.
- **How it works?** Uses Master-Minion model. Very fast communication.
- **Language:** YAML (State files)
- **Best for:** Large scale environments, real-time automation.
- **Advantage:** Very fast, handles thousands of servers.

Architecture (Master-Minion)



Example State File (YAML)

```
apache:
  pkg.installed
```

Quick Comparison

Agent Required

Language

Architecture

Easy to Learn

Speed

Best Use

Ansible

No

YAML

Agentless

★★★★★

Fast

Automation, simple tasks

Puppet

Yes

Puppet DSL

Master-Agent

★★★

Moderate

Configuration management

Chef

Yes

Ruby

Client-Server

★★

Moderate

Complex automation

SaltStack (Salt)

Yes (mostly)

YAML

Master-Minion

★★★

Very Fast

Large scale automation



ANSIBLE ARCHITECTURE (AGENTLESS)

Ansible is an **agentless** automation tool. It uses **SSH** to connect to managed nodes and executes tasks without installing any agent.

MAIN COMPONENTS



1. CONTROL NODE

- Machine where Ansible is installed.
- Contains Ansible Engine, modules, inventory, and playbooks.
- Initiates and controls automation.



2. INVENTORY

- List of managed hosts (nodes).
- Can be static file or dynamic (from cloud, scripts, etc.).



3. MODULES

- Small programs used to perform tasks.
- Examples: copy, yum, service, user, ping, etc.



4. PLAYBOOKS

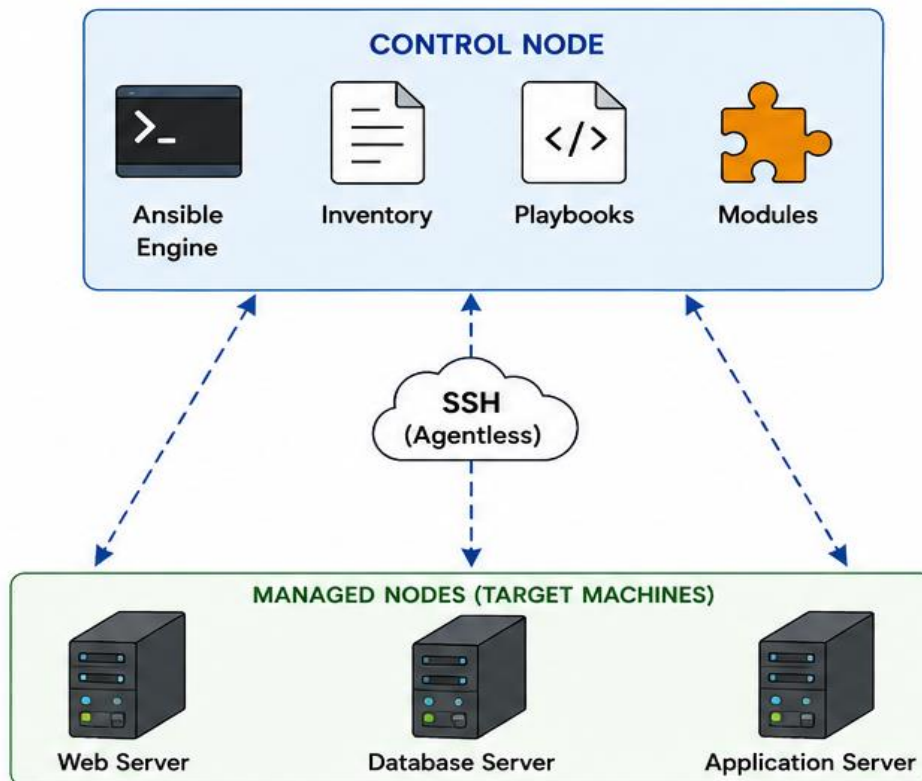
- Automation scripts written in YAML.
- Define tasks, hosts, roles, handlers, variables, etc.



5. CONNECTION (SSH)

- Ansible connects to managed nodes via SSH.
- No agent or software installation is required on nodes.

ARCHITECTURE DIAGRAM



WHAT ANSIBLE CAN DO



Configuration Management



Application Deployment



Cloud Provisioning



Task Automation



Monitoring & Orchestration

HOW IT WORKS (STEP BY STEP)

- User writes a playbook on the Control Node.
- Control Node reads the inventory (list of hosts).
- Ansible connects to managed nodes using SSH.
- Ansible modules are copied (if needed) and tasks are executed.
- Results are returned to the Control Node.
- No agent is installed on managed nodes.

EXAMPLE PLAYBOOK (YAML)

```
---
- hosts: webservers
  become: yes
  tasks:
    - name: Install Apache
      yum:
        name: httpd
        state: present
    - name: Start Apache
      service:
        name: httpd
        state: started
        enabled: yes
```

KEY BENEFITS

- ✓ Agentless – No software installation on nodes.
- ✓ Easy to learn and use (YAML based).
- ✓ Uses standard SSH – Secure and simple.
- ✓ Lightweight and fast.
- ✓ Works on Linux, Windows, Network devices, Cloud, Containers, etc.
- ✓ Ideal for configuration management, application deployment, and orchestration.

MEMORY TRICK:



Control Node = Brain



Inventory = List of Machines



Playbook = Instructions



Modules = Tools



SSH = Secret Passage



PUPPET ARCHITECTURE (MASTER-AGENT)

Puppet follows a Master-Agent architecture where one central Puppet Master manages multiple Puppet Agents (nodes).

MAIN COMPONENTS

1 PUPPET MASTER (PUPPET SERVER)



- Central server that stores all configurations.
- Compiles catalogs and sends to agents.
- Enforces desired state of all nodes.

2 PUPPET AGENT (NODE)



- Installed on every managed machine.
- Communicates with Master.
- Applies the catalog and reports the result.

3 MANIFEST



- Written in Puppet DSL.
- Describes the desired configuration.
- Example given below.

4 MODULES



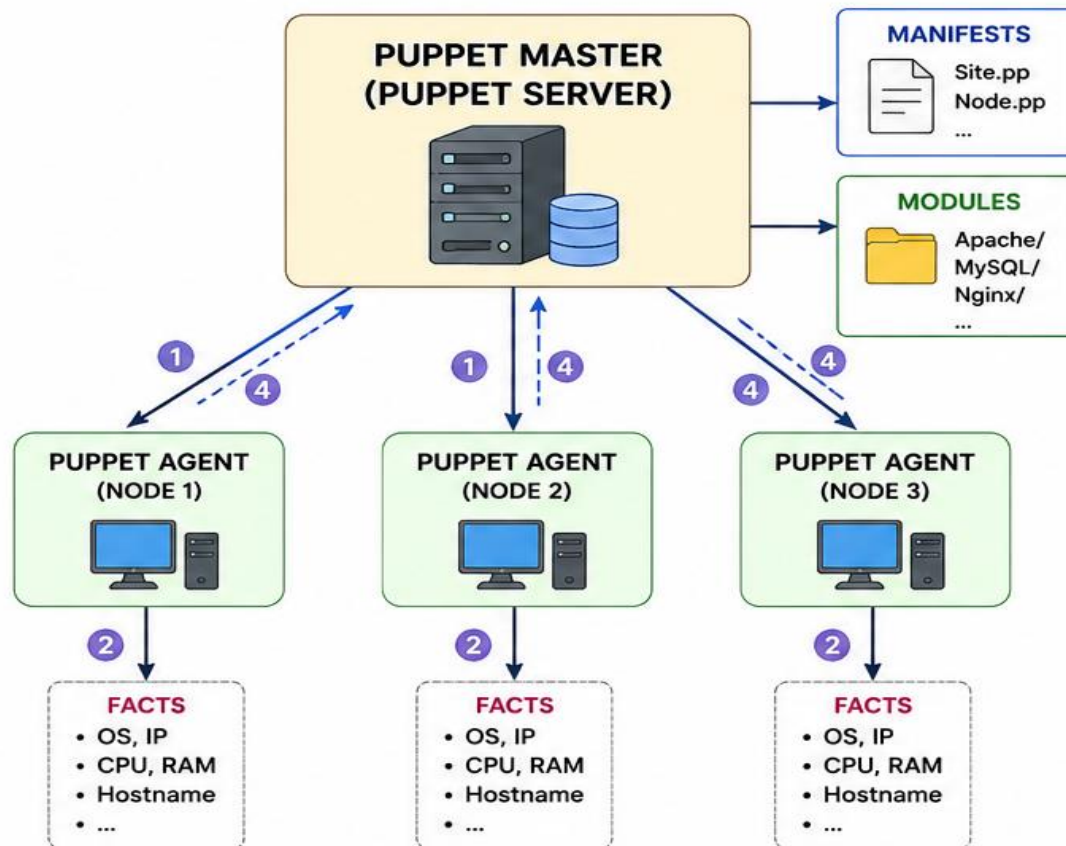
- Collection of manifests, templates, files, etc.
- Reusable code for managing applications and services.

5 FACTS (FACTER)



- System information collected by Facter.
- Includes OS, IP, CPU, RAM, hostname, etc.

ARCHITECTURE DIAGRAM



- 1 Agent sends Facts to Master
- 2 Master uses Facts and Manifests to compile Catalog
- 3 Master sends Catalog to Agent
- 4 Agent applies changes and reports result

HOW IT WORKS (STEP BY STEP)

1. Puppet Agent contacts the Puppet Master.
2. Agent sends its system information (Facts) to the Master.
3. Master uses Facts and Manifests to create a Catalog (list of resources).
4. Master sends the Catalog to the Agent.
5. Agent applies the configuration.
6. Agent reports the result back to Master.

EXAMPLE MANIFEST (PUPPET DSL)

```
package { 'apache2':
  ensure => installed,
}

service { 'apache2':
  ensure => running,
  enable => true,
}
```

KEY BENEFITS

- ✓ Keeps all systems in desired state.
- ✓ Automatically fixes configuration drift.
- ✓ Centralized management.
- ✓ Reusable modules save time.
- ✓ Improves security and compliance.

MEMORY TRICK:



Puppet Master = The Boss



Puppet Agent = The Worker



Manifest = Rule Book



Module = Toolkit



Facts = Identity Card

CHEF CONFIGURATION MANAGEMENT

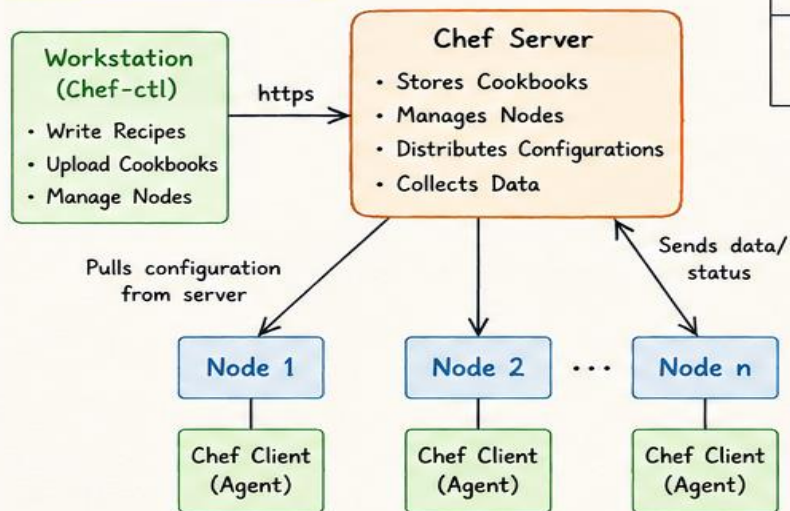
1. INTRODUCTION

Chef is an open-source automation and configuration management tool used to deploy, configure and manage infrastructure as code. It helps in maintaining consistency across environments.

Key Features:

- Infrastructure as Code
- Automation of repetitive tasks
- Scalability and flexibility
- Agent-based architecture
- Supports multiple platforms

2. ARCHITECTURE



* Chef follows a pull-based model. Nodes pull configurations from the Chef Server.

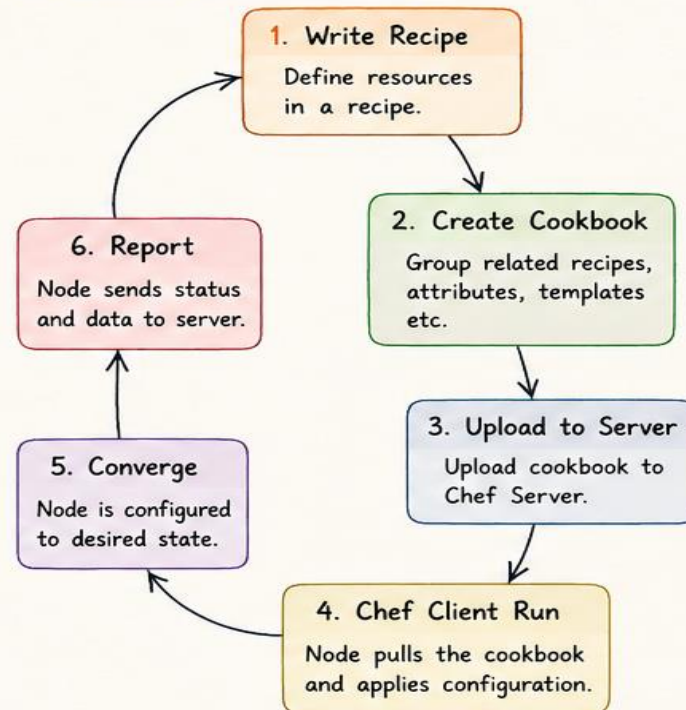
3. CORE CONCEPTS & VOCABULARY

Concept	Description
Node	A server or machine managed by Chef. It runs Chef Client.
Chef Server	Centralized server that stores cookbooks, manages nodes and distributes configurations.
Cookbook	A collection of recipes, attributes, templates and other files.
Recipe	A set of instructions written in Ruby DSL that defines how a system should be configured.
Resource	A declaration of a system component (file, package, service, user, etc.) to be managed.
Attribute	Data that can be used in recipes to make them dynamic.

5. COMPONENTS

- **Chef Workstation** : Used by developers/ops to create cookbooks and manage Chef Server.
- **Chef Server** : Stores cookbooks, manages nodes and handles communication.
- **Chef Client** : Installed on nodes; pulls configuration from server and applies it.

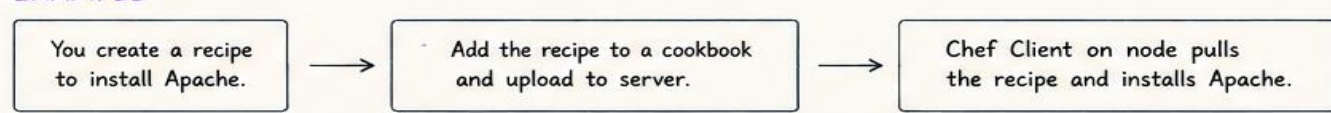
4. CHEF WORKFLOW



6. BENEFITS

- Ensures consistency across environments
- Reduces manual errors
- Saves time through automation
- Easy to scale infrastructure
- Supports version control and collaboration

EXAMPLE



SALT CONFIGURATION MANAGEMENT

Automate • Configure • Orchestrate

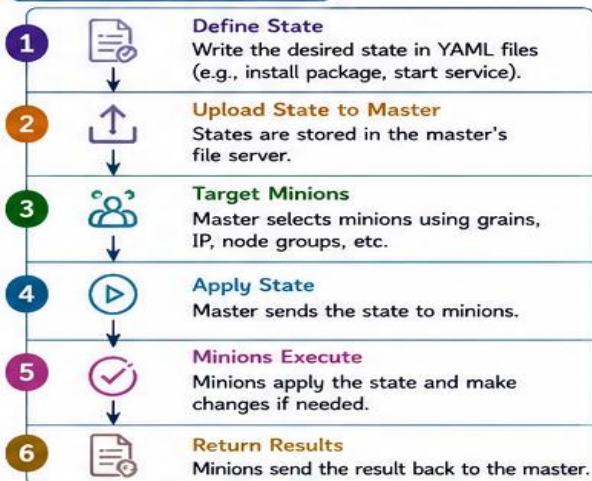
1. INTRODUCTION

Salt (Simple, Fast, Scalable) is an open-source configuration management and remote execution tool. It is used to manage infrastructure, automate tasks, and maintain system states consistently across servers.

Key Features

- Agentless architecture (uses SSH by default)
- Fast and scalable
- Event-driven and real-time
- Supports automation and orchestration
- Cross-platform (Linux, Windows, macOS)
- Extensible with modules and states

4. SALT WORKFLOW



7. EXAMPLE: INSTALL NGINX USING A STATE



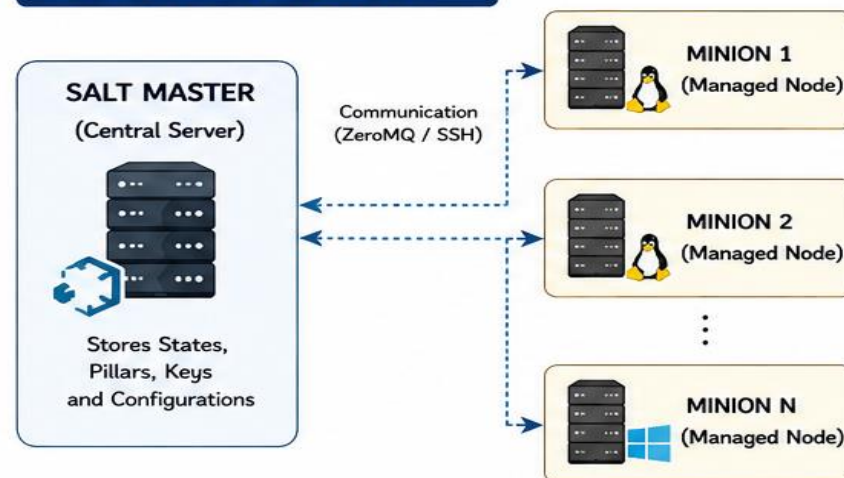
2. CORE CONCEPTS & VOCABULARY

Term	Description
Master	The central server that manages all operations and communicates with minions.
Minion	The managed nodes (systems) that accept commands from the master.
State	Defines the desired state of a system using YAML files.
Pillar	Global data management system for storing configuration data.
Grains	Static information about a minion (OS, IP, etc.) used for targeting.
Modules	Functions used by Salt to perform tasks (e.g., package.install, service.start).
Beacons	Detect and report changes or events from minions.
Reactors	Trigger actions in response to events.

5. SALT COMPONENTS

- **Master**
Central management server. Handles authentication, state compilation, job scheduling and results.
- **Minion**
Installed on managed nodes. Accepts commands and maintains the desired state.
- **States**
YAML files that define the desired configuration of systems.
- **Pillar**
Encrypted key-value data used to manage global configuration and sensitive data.
- **Salt Key**
Manages authentication keys between master and minions.
- **File Server**
Stores states, pillar data, and other distribution files.

3. SALT ARCHITECTURE



6. BASIC SALT COMMANDS (Examples)

Command	Description
<code>salt '*' test.ping</code>	Check connectivity with all minions
<code>salt '*' sysinfo.system</code>	Get system information
<code>salt 'web*' pkg.install nginx</code>	Install nginx package on minions whose ID starts with 'web'
<code>salt '*' service.start httpd</code>	Start httpd service on all minions
<code>salt '*' state.apply <state_name></code>	Apply a state to all minions
<code>salt '*' state.highstate</code>	Apply all states (top file) to all minions

8. BENEFITS

- ✓ Fast and efficient
- ✓ Agentless (no software required on minions)
- ✓ Highly scalable
- ✓ Easy automation and orchestration
- ✓ Powerful targeting using grains
- ✓ Real-time event handling
- ✓ Open source and extensible

NGINX (pronounced "engine-ex") is a high-performance, open-source web server software program

Introduction to Ansible

- ▶ **Ansible** is an **open-source** software that is used to **configure systems, manage IT infrastructure, deploy software, and system updates**. It is a **command-line** software that is written in Python.
- ▶ **Ansible** uses a simple and easy-to-understand language, that language is called **YAML** for writing its configuration files. **YAML** is making it accessible even to those who are new to the coding.
- ▶ **Ansible** works by connecting to your servers. Those servers are connected using **SSH(Secure Shell)** and executing tasks remotely. This means you don't need to install any other software on your servers, which can save your time.
- ▶ One of the biggest strengths of Ansible it does not require an agent, which makes it lightweight and easy to set up. It also has a large and active community that provides a large amount of support and resources.
- ▶ Ansible is great for automating repetitive tasks, such as **software installation, configuration management, and application deployment**.

► **Advantages of Ansible:**

- **Agentless:** No need to install agents on managed nodes.
- **YAML Syntax:** Playbooks are written in YAML, which is easy to understand.
- **Push-Based:** Configurations are pushed from a central control machine to remote nodes.
- **Simplicity:** Known for its straightforward setup and usage, suitable for beginners.
- **Community Support:** Benefits from a large and active community providing extensive documentation and support resources.

► **Disadvantages of Ansible:**

- **Limited Granularity:** May lack the granularity and control provided by other tools like Chef.
- **Less Scalable:** Not as well-suited for managing extremely large and complex infrastructures compared to Chef.
- **State Management:** Limited capabilities for managing the state of systems over time.
- **Performance Overhead:** Push-based architecture may lead to performance issues in large-scale deployments with numerous nodes.

Introduction to Puppet

- ▶ It is an open-source software program configuration administration and deployment tool.
- ▶ It's most frequently used on Linux and Windows to pull the strings on a couple of utility servers at once.
- ▶ It is free for use and open to change and customization. You get a complete device with core CM abilities and performance.

► **Advantages of Puppet**

- The following are some advantages of using Puppet :
- It is easy to install.
- It has Command Line Interface and Graphical User Interface.
- It has a support for all major Operating Systems.

► **Disadvantages of Puppet**

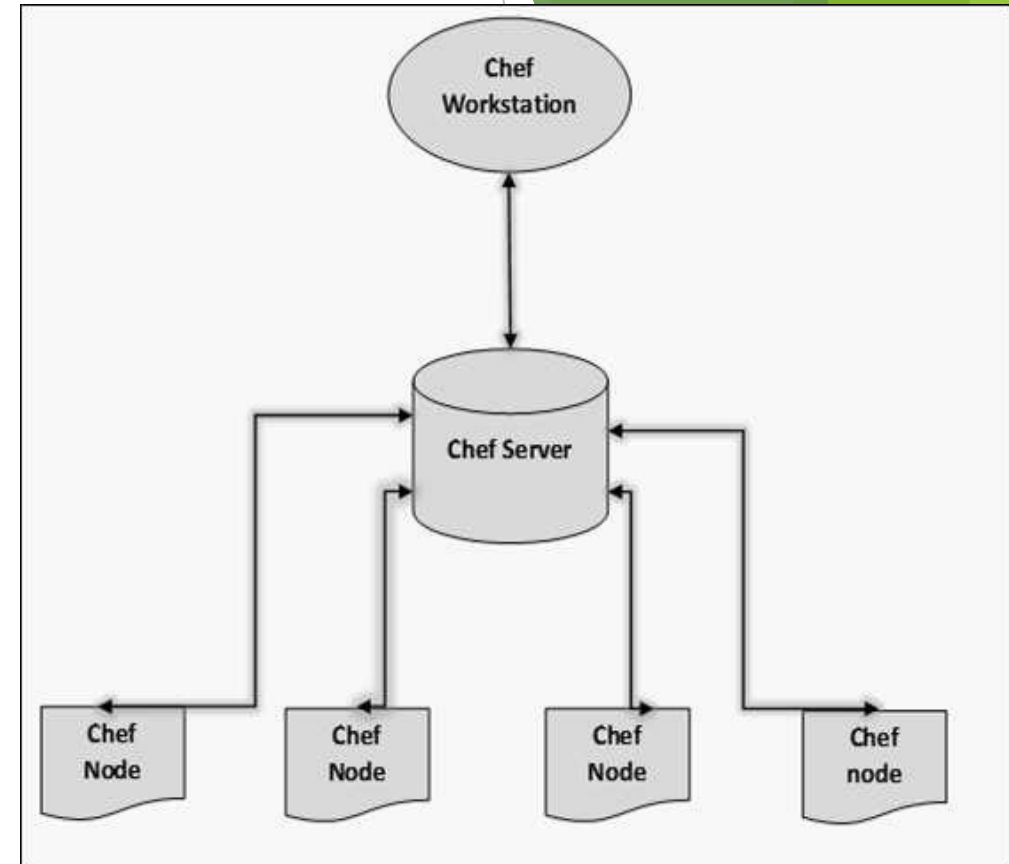
- The following are some disadvantages of using Puppet:
- Quite slow to respond.
- It does not have lifecycle management.
- Tough for beginners to learn.

Introduction to Chef

- ▶ **Chef** uses a **declarative approach** to configuration management. Where you specify the desired state of your infrastructure and **Chef** ensures that systems match that state.
- ▶ It uses Ruby, a **domain-specific language** that may take some time to master for individuals who are not experienced with programming.
- ▶ Chef is designed on a client-server model.
- ▶ Configuration data is stored on a Chef server and distributed to nodes/servers through the agents or Chef client. This increases your visibility and control over your infrastructure, particularly in large and more complex contexts.
- ▶ Chef works best in settings where you want to ensure uniformity throughout your infrastructure and require fine-grained control over configurations.

Chef's Components

- **Chef Server** – The main computer that stores all the configurations (recipes and cookbooks).
- **Chef Client (Node)** – The software installed on each server/computer.
 - It contacts the Chef Server, downloads the instructions, and applies them.
- **Workstation** – The developer's computer where recipes and cookbooks are created.



► Advantages of Chef

- **Client-Server Architecture:** Nodes communicate with a central Chef server.
- **Declarative Language:** Uses a domain-specific language(DSL) based on Ruby.
- **Pull-Based:** Nodes periodically retrieve configurations from the central server.
- **Scalability:** Designed for managing large-scale deployments and complex infrastructures.
- **Extensibility:** It offers a high ecosystem of cookbooks, recipes, and community-contributed resources for customization.

► Disadvantages of Chef:

- **Learning Curve:** Steeper learning curve, particularly for those unfamiliar with Ruby or DSL-based configuration management.
- **Agent-Based:** Requires installation and management of agents on remote nodes, adding complexity to setup and maintenance.
- **Resource Intensive:** Pull-based model may consume more resources, especially in environments with a large number of nodes.
- **Complexity:** Managing a Chef server and maintaining consistency across configurations can be complex and time-consuming.

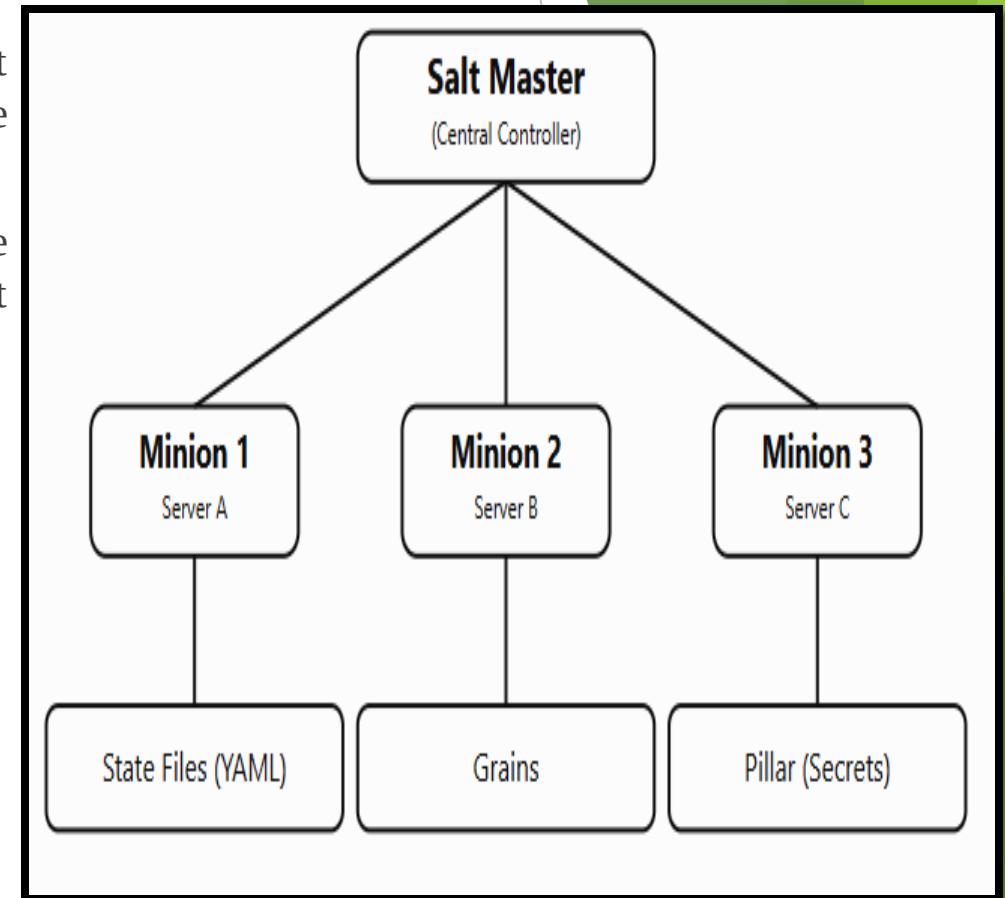
Salt(Stack)

- ▶ Saltstack is open-source software.
- ▶ SaltStack is a powerful and flexible configuration management system.
- ▶ It automates various code deployment and system admin tasks and eliminates the manual process.
- ▶ In this way, it can reduce errors that IT organizations face while configuring their systems.
- ▶ It runs from the command line interface. The paid version of SaltStack provides a GUI for Windows and MacOS.
- ▶ **Features:**
 - ▶ Powerful and flexible configuration management.
 - ▶ Automation of code deployment and system admin tasks.
 - ▶ Open-source software with a command-line interface.
 - ▶ GUI is available in the paid version for Windows and MacOS.

Architecture of Salt

- ▶ It traditionally follows a **Master-Minion architecture**, where the Salt Master controls Salt Minions.
- ▶ Salt provides remote command execution, configuration management through **State files**, system information through **Grains**, and secure configuration data through **Pillar**.
- ▶ It is widely used in DevOps for automated server configuration, software deployment, infrastructure management, and maintaining consistent system configurations.

Component	Meaning
Salt Master	The boss that sends instructions. [central controller]
Salt Minion	The worker that executes commands [A Salt Minion is a small agent installed on each managed server]
State Files	The rule book written in YAML [desired configuration of a system]
Grains	The Minion's identity card (OS, CPU, RAM)
Pillar	The secret locker for passwords and keys [stores sensitive and secure information]



SaltStack is Used in DevOps

SaltStack is popular because it automates repetitive administration tasks.

It is used for:

- ▶ **Automated server configuration** – Configure many servers at once.
- ▶ **Software deployment** – Install or update applications on multiple machines.
- ▶ **Infrastructure management** – Manage cloud and data center servers from one place.
- ▶ **Maintaining consistency** – Ensure every server has the same configuration.
- ▶ **Remote command execution** – Run commands instantly on thousands of servers.

Advantages of Salt

▶ 1. Fast Execution

- ▶ Salt is designed for high-speed communication and remote execution.

▶ 2. Scalable

- ▶ It can be used to manage a large number of systems.

▶ 3. Automation

- ▶ Routine administration tasks can be automated.

▶ 4. Centralized Management

- ▶ Multiple servers can be managed from a central Salt Master.

▶ 5. Desired-State Configuration

- ▶ Salt States help maintain consistent system configurations.

▶ 6. Remote Execution

- ▶ Commands can be executed on multiple systems simultaneously.

▶ 7. Flexible Targeting

- ▶ Specific servers can be selected based on IDs, patterns, or system characteristics.

▶ 8. Infrastructure as Code

- ▶ Configuration can be represented in reusable, version-controlled files.

Limitations of Salt

- ▶ Traditional Salt architecture requires installing and managing minions.
- ▶ Configuration can become complex in very large environments.
- ▶ Administrators need knowledge of YAML and Salt concepts.
- ▶ Master-minion architecture introduces additional infrastructure.
- ▶ Secure key and access management must be properly maintained.

Comparison of Ansible, Puppet, Chef, and SaltStack

Feature	Ansible	Puppet	Chef	SaltStack (Salt)
Developed by	Ansible Inc. / Red Hat	Puppet	Chef Software	Salt Project
Architecture	Control node & managed nodes	Master-Agent	Server-Client	Master-Minions
Agent Required	No	Yes	Yes	Usually yes
Communication	SSH / WinRM	Puppet protocol	HTTPS	ZeroMQ / TCP-based communication
Configuration Language	YAML	Puppet DSL	Ruby-based DSL	YAML / Jinja

Comparison of Ansible, Puppet, Chef, and SaltStack cont....

Feature	Ansible	Puppet	Chef	SaltStack (Salt)
Configuration Files	Playbooks	Manifests	Recipes / Cookbooks	State (SLS) files
Typical Model	Push	Pull	Pull	Push / event-driven; commonly Master-Minion
Ease of Learning	Easy	Moderate	Difficult	Moderate
Remote Execution	Yes	Limited compared with Salt	Yes	Very strong
Scalability	High	Very high	Very high	Very high
Best Known For	Simple automation & deployment	Continuous configuration enforcement	Infrastructure as Code	Fast remote execution & automation

ANSIBLE

- ▶ INTRODUCTION
- ▶ FEATURES
- ▶ ARCHITECTURE
- ▶ USE CASES
- ▶ WITH DevOps
- ▶ PLAYBOOK
- ▶ ROLES
- ▶ BENEFITS
- ▶ LIMITATION
- ▶ WRITING ANSIBLE PLAYBOOKS USING YAML

ANSIBLE

- ▶ Ansible is an open-source automation tool that helps in simplifying IT process.
- ▶ It does this by allowing users to define automation tasks using playbook YAML.
- ▶ It also helps in managing systems without installing agents on the target machine as it is agentless in nature.

ANSIBLE

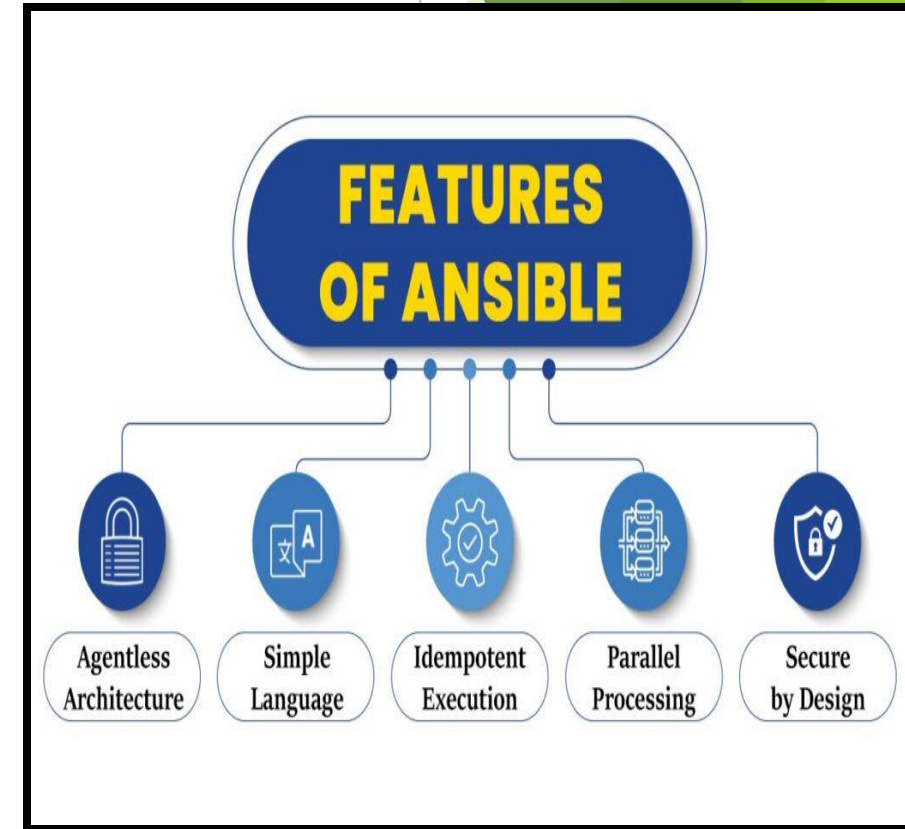
- ▶ Ansible is an **open-source IT engine** that automates application deployment, cloud provisioning, intra-service orchestration, and other IT tools.
- ▶ It is an **automation and orchestration tool** popular for the following reasons:
 - ▶ Simple to Install.
 - ▶ Free and open source.
 - ▶ Lightweight and consistent.
 - ▶ OpenSSH security features make it very secure.

- ▶ Ansible can be used to deploy the software on different servers at a time without human interaction.
- ▶ Ansible can also be used to configure the servers and create user accounts.
- ▶ Ansible is an agent-less software which means there is no need to install the software in the nodes which means you need to do the SSH to connect the nodes to perform the required operations on the servers.
- ▶ Ansible playbooks can be written very easily they will be like plain english and Ansible is developed using Python language.
- ▶ There is no need for any knowledge of programming.
- ▶ A single ansible control node can manage thousands of the nodes.

ANSIBLE FEATURES

<https://www.pynetlabs.com/what-is-ansible-and-what-is-it-used-for/>

- Agentless Architecture:** Ansible connects through SSH, which means no software runs on managed servers. The control node handles all the logic and execution.
- Simple Language:** Ansible makes use of YAML for configuration. Tasks mentioned in YAML are simple, plain English.
- Idempotent Execution:** Tasks run multiple times safely. Systems reach the desired state without breaking.
- Parallel Processing:** Ansible can easily control many servers at once. All the tasks can run simultaneously.
- Secure by Design:** Ansible encrypts passwords with Vault and uses SSH keys. Credentials stay protected in playbooks.





ANSIBLE ARCHITECTURE (AGENTLESS)

Ansible is an **agentless** automation tool. It uses **SSH** to connect to managed nodes and executes tasks without installing any agent.

MAIN COMPONENTS



1. CONTROL NODE

- Machine where Ansible is installed.
- Contains Ansible Engine, modules, inventory, and playbooks.
- Initiates and controls automation.



2. INVENTORY

- List of managed hosts (nodes).
- Can be static file or dynamic (from cloud, scripts, etc.).



3. MODULES

- Small programs used to perform tasks.
- Examples: copy, yum, service, user, ping, etc.



4. PLAYBOOKS

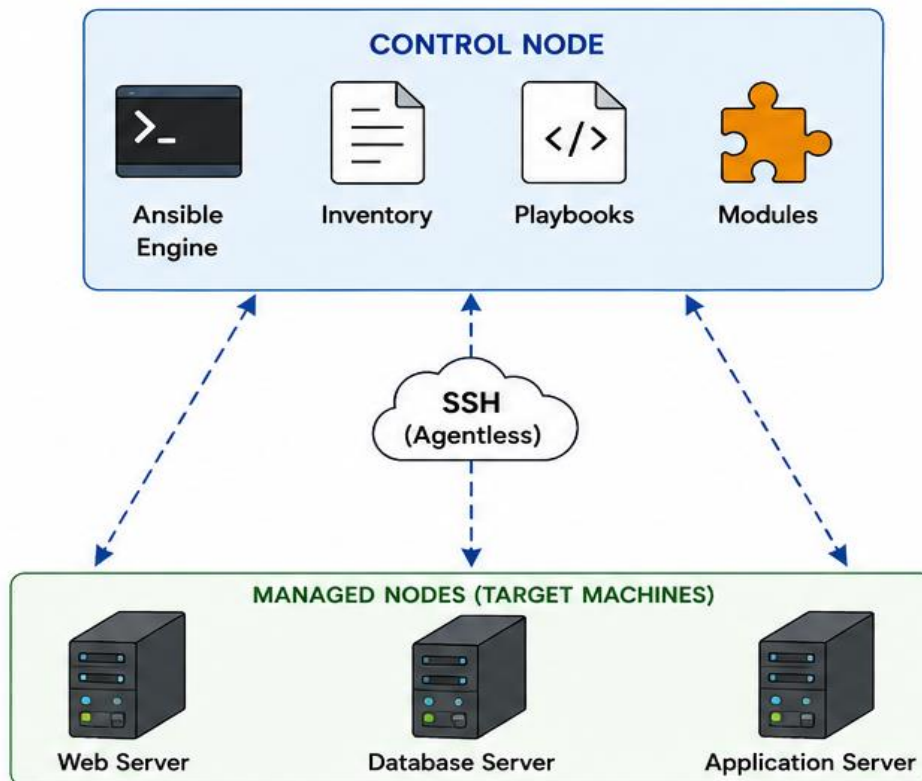
- Automation scripts written in YAML.
- Define tasks, hosts, roles, handlers, variables, etc.



5. CONNECTION (SSH)

- Ansible connects to managed nodes via SSH.
- No agent or software installation is required on nodes.

ARCHITECTURE DIAGRAM



WHAT ANSIBLE CAN DO



Configuration Management



Application Deployment



Cloud Provisioning



Task Automation



Monitoring & Orchestration

HOW IT WORKS (STEP BY STEP)

- User writes a playbook on the Control Node.
- Control Node reads the inventory (list of hosts).
- Ansible connects to managed nodes using SSH.
- Ansible modules are copied (if needed) and tasks are executed.
- Results are returned to the Control Node.
- No agent is installed on managed nodes.

EXAMPLE PLAYBOOK (YAML)

```
---
- hosts: webservers
  become: yes
  tasks:
    - name: Install Apache
      yum:
        name: httpd
        state: present
    - name: Start Apache
      service:
        name: httpd
        state: started
        enabled: yes
```

KEY BENEFITS

- ✓ Agentless – No software installation on nodes.
- ✓ Easy to learn and use (YAML based).
- ✓ Uses standard SSH – Secure and simple.
- ✓ Lightweight and fast.
- ✓ Works on Linux, Windows, Network devices, Cloud, Containers, etc.
- ✓ Ideal for configuration management, application deployment, and orchestration.

MEMORY TRICK:



Control Node = Brain



Inventory = List of Machines



Playbook = Instructions

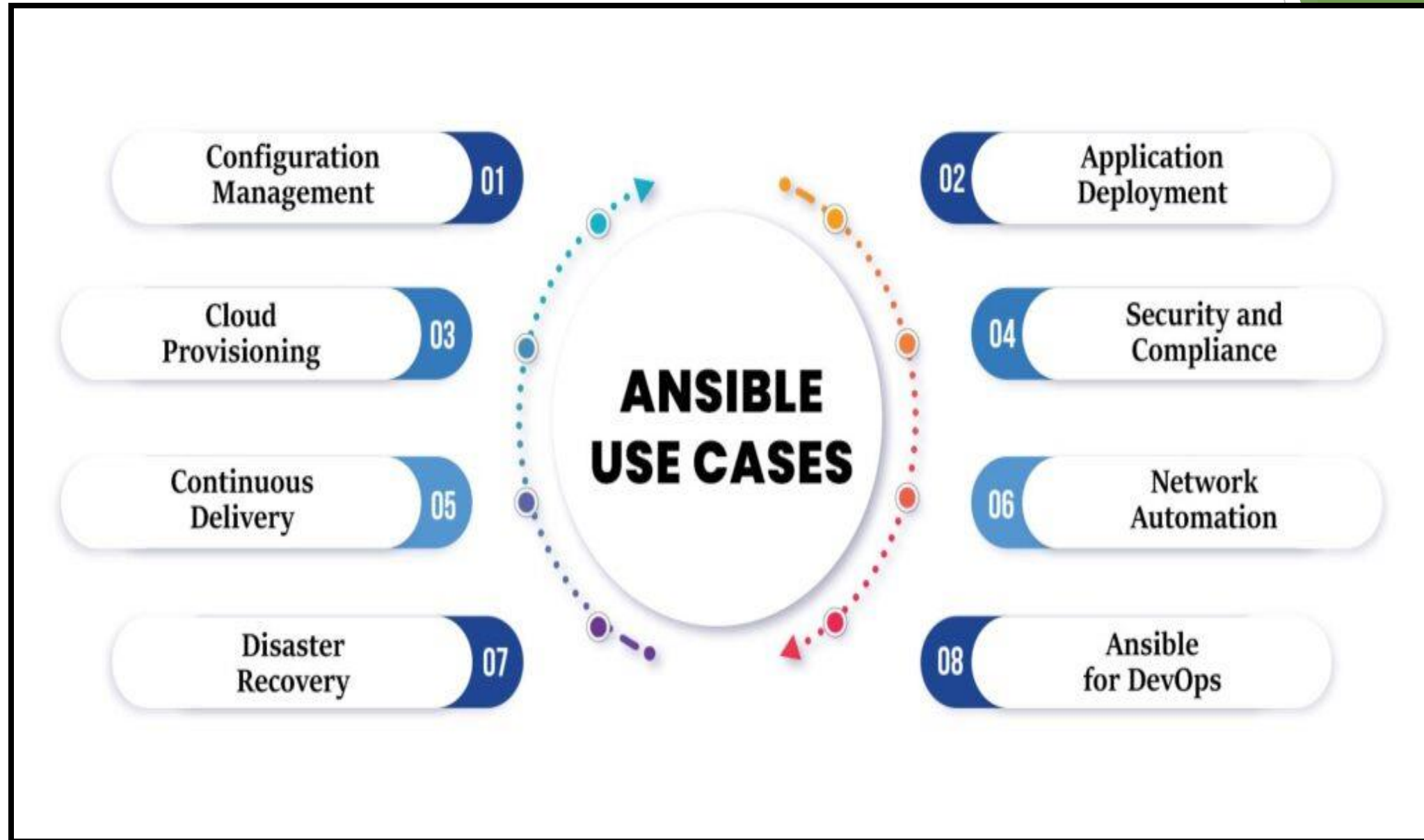


Modules = Tools



SSH = Secret Passage

ANSIBLE USE CASES



► **Configuration Management**

- Ansible shines at keeping servers consistent. You write playbooks that define how systems should look.
- Then Ansible makes it happen. Need Apache on 50 servers? One playbook does it all.
- If you change your mind about a setting? Update the playbook and then rerun it.
- Every server matches perfectly.

► **Application Deployment**

- Deploying apps manually takes forever. Ansible automates the whole process. It copies files, sets permissions, and restarts services.
- Your app goes from development to production smoothly.

► **Cloud Provisioning**

- Building cloud infrastructure used to mean clicking through web interfaces. Now Ansible talks directly to AWS, Azure, or Google Cloud.
- It spins up servers, configures networks, and sets up storage. Everything happens through code.

► **Security and Compliance**

- Keeping systems secure requires constant updates. Ansible patches servers, rotates passwords, and configures firewalls.
- It runs compliance checks, too. When auditors ask about your security setup, you show them the playbooks.

► **CI/CD Workflows**

- Modern teams deploy code multiple times daily. Ansible fits perfectly into CI/CD pipelines. It handles the deployment piece, working with tools like Jenkins or GitLab. Code changes flow automatically from the repository to production.

► **Network Automation**

- Network devices need updates, too. Ansible configures routers, switches, and firewalls from major vendors. It backs up configs, pushes changes, and validates settings. Network teams save hours by automating repetitive tasks. One playbook can update VLANs across your entire network.

► **Disaster Recovery**

- When systems crash, every minute counts. Ansible helps rebuild infrastructure fast. Your playbooks become recovery documentation. They restore servers, databases, and applications in the right order. What took hours manually now happens in minutes. Regular disaster drills become simple, just run the playbooks and verify everything works.

Ansible for DevOps

- ▶ Ansible is widely used in DevOps because it helps teams automate deployment and infrastructure tasks.
- ▶ DevOps teams use Ansible to:
 - ▶ deploy applications
 - ▶ configure servers
 - ▶ manage environment consistency
 - ▶ run repeatable release tasks
 - ▶ automate rollback steps
 - ▶ integrate deployment workflows with CI/CD tools
- ▶ **Ansible playbooks can be stored in Git.**
- ▶ This allows teams to review, version, and reuse automation just like application code.
- ▶ This makes Ansible useful for teams that want faster releases and fewer manual deployment errors.

ROLES

Ansible Roles help in organizing playbooks and files.

Roles group related tasks, variables, and files together.

They make sharing code easier. You can download roles from Ansible Galaxy or create your own.

Below, we have discussed each component of Ansible architecture in a tabular form.

Component	Role in Ansible
Control Node	The machine where Ansible commands and playbooks run
Managed Nodes	Servers, devices, or systems managed by Ansible
Inventory	A file or source that lists managed nodes
Playbooks	YAML files that define automation tasks
Modules	Small units of code that perform tasks
Plugins	Extensions that add extra Ansible behavior
Collections	Packages that contain modules, plugins, roles, and playbooks
Roles	Reusable structure for organizing playbooks

Ansible follows a push-based model in most common setups. The control node pushes instructions to managed nodes and runs tasks remotely. This agentless model is one of Ansible's biggest advantages. Managed nodes do not usually need Ansible installed on them.

Benefits of Ansible

- ▶ Ansible offers many benefits to IT and team members involved in network automation.
- ▶ **Agentless setup**
- ▶ Ansible doesn't require agents on the majority of managed nodes. It generally utilizes SSH for Linux and [network devices](#), as well as WinRM for Windows. This makes the deployment process simpler than many automation tools based on agents.
- ▶ **Simple syntax for YAML**
- ▶ Ansible playbooks make use of YAML. YAML is easy to read and understand for beginners than many programming languages. This is beneficial to system administrators, DevOps engineers, and network engineers to automate their work faster.
- ▶ **Idempotent execution**
- ▶ Ansible aims to bring systems into that desired state. If the desired state already exist, Ansible does not make unnecessary changes. This makes the repeated execution of playbooks more secure and predictable.
- ▶ **Large ecosystem**
- ▶ Ansible can be used with a variety of operating systems, cloud platforms, and network devices via collections and modules. This makes Ansible beneficial across security, cloud, infrastructure, and networking teams.
- ▶ **Good for teams**
- ▶ Playbooks can be saved in Git and reviewed by team members and reused across different environments. This supports better collaboration and less manual errors

Limitations of Ansible

- ▶ Ansible is an extremely powerful tool; however, beginners should be aware of its limitations.
- ▶ **YAML errors can break playbooks**
- ▶ YAML is readable, but mistakes in indentation could cause errors. Beginners should verify syntax before using playbooks.
- ▶ **Large environments need planning**
- ▶ As automation expands the playbooks get become messy. Teams must use roles, collection, variables, and roles to ensure that automation is organized.
- ▶ **Real-time event handling needs extra tools**
- ▶ Basic Ansible is usually executed either manually or by scheduling workflows. Event-driven use cases require additional tools, like Event-Driven Ansible, or platform-level tools.
- ▶ **Secrets need proper handling**
- ▶ API keys and passwords should not be stored in plaintext. Ansible Vault or an appropriate secrets-management system must be utilized.
- ▶ **Network automation requires device-specific knowledge**
- ▶ Ansible can automate routers and switches, but engineers need to be aware of networking knowledge. A playbook can't replace the knowledge of VLANs, ACLs, routing or the behavior of devices.

Writing Ansible Playbooks using YAML

▶ Playbooks (YAML Automation)

▶ **Structure:** YAML files containing "plays" that map managed hosts to specific roles and tasks.

▶ **Execution:** Runs tasks in sequential order across multiple target machines simultaneously.

▶ **Purpose:** Automates software installation, file transfers, service management, and configuration states.

•Plugins (Extended Capability)

•**Definition:** Modularity code blocks that extend Ansible's out-of-the-box engine features.

•**Functions:** Enhances connection handling, inventory data lookups, logging callbacks, and custom filters.

•**Flexibility:** Supports a wide variety of native built-in plugins as well as custom user-created code.

•Inventory (Host Management)

•**Definition:** A centralized text file listing all managed target endpoints, IPs, and server details.

•**Targeting:** Group hosts together to assign variables and execute tasks dynamically across multiple nodes.

•**Network Operations:** Scales to map infrastructure hardware like switches, routers, firewalls, and load balancers.

- ▶ An **Ansible Playbook** is the master recipe.
- ▶ It is a simple text file that tells computers exactly how to update themselves, install apps, or adjust their settings all at the same time.

The Three Magic Rules of Ansible

1.It Speaks Plain English: You do not need to be a coding genius. Playbooks are written in **YAML**, a format that reads like a structured shopping list.

2.It is Agentless (No Stowaways): Ansible does not require installing heavy background software inside the remote computers. It simply knocks on the front door using standard secure network connections (**SSH**), delivers the instructions, and leaves.

3.It is Idempotent (The Smart Door Rule): If you tell a robot to “*close an open door*,” it closes it. If you run the script again and the door is already closed, the robot looks at it, does nothing, and says ok. It never breaks a working machine by repeating tasks unnecessarily.

Playbooks are organized into containers that fit inside each other, just like nested dolls:

- **The Playbook (The Book)**: The whole text file.
- **The Play (The Chapter)**: Defines **who** is targeted (e.g., hosts: webserver) and how to log in (Eg - *become: true* for administrative powers).
- **The Task (The Instruction)**: A single step on the checklist (e.g - "Install a game").
- **The Module (The Tool)**: The built-in worker script that knows exactly how to perform the action behind the scenes (like apt for software or copy for files).
- **---**: Written on the very first line to prove to the computer that this is an official configuration document.
- **- name:** - A human description label. Ansible prints this out on your screen while it works so you can watch what step it is running.
- **state: present**: Means *"If it is missing, install it. If it is already there, skip it."*

RUSSION DOLL ANALOGY

- The Playbook (The Largest Doll):** The overall physical text file (.yml). It holds everything together.
- The Play (The Next Doll Inside):** Defines the environment context—**who** is being targeted (e.g., hosts: database_servers) and security rights (become: true).
- The Task (The Next Doll Inside):** An individual item on a chronological checklist (e.g., "Step 1: Install Python").
- The Module (The Smallest Doll):** The highly specific, hidden action script doing the hard work (e.g., apt, yum, or copy).

YAML Basics

- All YAML files begin with `---` and end with `...`.
- This is part of the YAML format and indicates the start and end of a document.

All members of a list are lines beginning at the same indentation level starting with a `"- "` (a dash and a space):

```
---  
# A list of tasty fruits  
- Apple  
- Orange  
- Strawberry  
- Mango  
...
```

A dictionary is represented in a simple **key: value** form (the colon must be followed by a space):

```
# An employee record  
martin:  
  name: Martin  
  D'vloper  
  job: Developer  
  skill: Elite
```

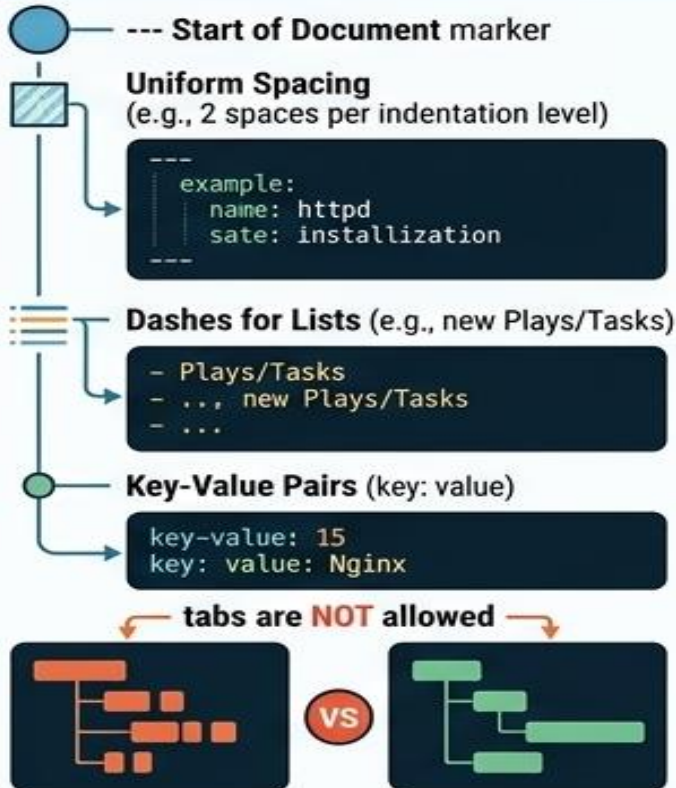
More complicated data structures are possible, such as lists of dictionaries, dictionaries whose values are lists or a mix of both:

```
# Employee records  
- martin:  
  name: Martin D'vloper  
  job: Developer  
  skills:  
    - python  
    - perl  
    - pascal  
- tabitha:  
  name: Tabitha Bitumen  
  job: Developer  
  skills:  
    - lisp  
    - fortran  
    - erlang
```

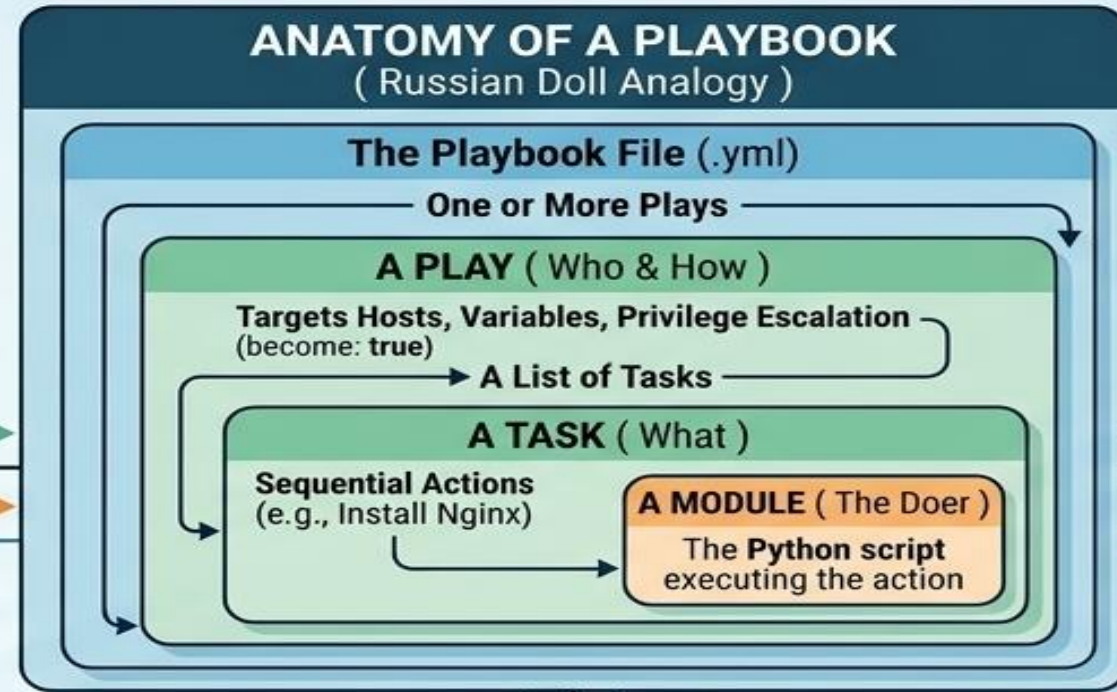
UNPACKING AN **ANSIBLE PLAYBOOK**: EXPLANATION

A HUMAN-READABLE GUIDE TO IT AUTOMATION

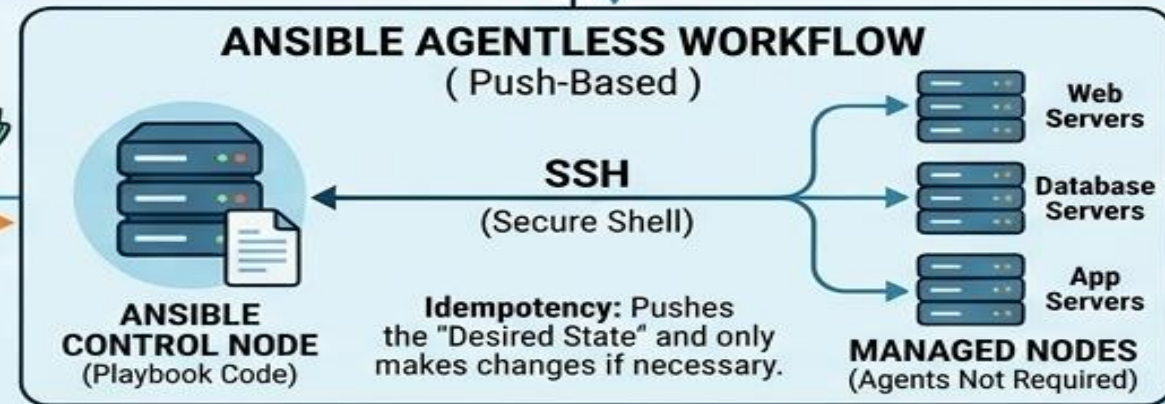
UNDERSTANDING YAML SYNTAX & KEY RULES



ANATOMY OF A PLAYBOOK (Russian Doll Analogy)



ANSIBLE AGENTLESS WORKFLOW (Push-Based)



WRITING AN ANSIBLE PLAYBOOK USING YAML: A STEP-BY-STEP VISUAL GUIDE

AN ANSIBLE PLAYBOOK (.yml file)



→ Playbook
→ Plays
→ Tasks
→ Modules

1. GATHER COMPONENTS

Playbook Code

Inventory File

Ansible Engine

2. WRITE PLAYBOOK IN EDITOR

```
---  
- name: Ensure  
  copy:  
    content: "<h1>wel  
    owner: www-data  
    group: www-data  
    code: '0644'
```

YAML Rule:
uniform spaces
(No Tabs!)

PLAYBOOK EDITOR: setup_webserver.yml

```
1  ---  
2  # The overall file (The Playbook)  
3  - name: Configure Web Servers  
4    hosts: webserver # The Play targets this host group  
5    become: true     # Give Ansible admin (root) access  
6  
7  # Ordered list of sequential actions (The Tasks)  
8  tasks:  
9    - name: Ensure Apache package is present  
10      yum: # The Module handling the action  
11        name: httpd  
12        state: present # Desired State: Idempotent  
13  
14    - name: Create index.html homepage  
15      copy: # A file manipulation Module  
16        content: "<h1>welcome to Automated DevOps!</h1>\n"  
17        dest: /var/www/html/index.html  
18        owner: www-data  
19        group: www-data  
20        code: '0644'  
21  
22    - name: Start Apache service and enable on boot  
23      service: # The Module for services  
24        name: httpd  
25        state: started  
26        enabled: true
```

Start of Document
marker (---)

Declarative
Parameters

A Module
(The action
block)

The Module
for services

3. DEPLOY & ORCHESTRATE

SSH (Secure Shell)

ANSIBLE CONTROL NODE

Idempotency:

Checks desired state,
only makes necessary
changes.

✓ ok: 2
✓ changed

✓ ok: 2

✓ ok: 2
✓ changed: 1

✓ ok: 1

MANAGED NODES
Web Servers

COMPLETE CONFIGURATION: SCRIPT DEPLOYMENT SUCCESSFUL

Automation

DevOps

Speed

EXAMPLE PLAYBOOK

```
1 ---
2 - name: Install Apache Web Server
3   hosts: webservers
4   become: yes
5
6   tasks:
7     - name: Install Apache
8       apt:
9         name: apache2
10        state: present
11     - name: Start Apache Service
12       service:
13         name: apache2
14         state: started
15         enabled: yes
```

INVENTORY EXAMPLE

```
[webservers]
192.168.1.10
192.168.1.11
```

YAML TIP

YAML is sensitive to indentation.
Use proper spaces (not tabs).
Indentation shows relationship.

PLAYBOOK STRUCTURE SUMMARY

Play	One complete set of instructions (the whole recipe).
Hosts	Target machines or groups where tasks will run.
Become	Get administrator (sudo) permission.
Tasks	List of step-by-step instructions.
Modules	Built-in tools that perform actual actions.
YAML	Indentation keeps everything organized.



1

Marks the beginning of YAML document.



Analogy
Opening a new recipe book.

2

- **name:** Install Apache Web Server
Gives a name to the play. For humans only.



Analogy
Recipe Title.

3

hosts: webservers
Tells Ansible which machines (group) should perform these tasks.



Analogy
Teacher gives homework to Class A only.

4

become: yes
Provides administrator (sudo) privileges.



Analogy
Getting a driving license to do important tasks.

5

tasks:
Starts the list of actions. All tasks will run one by one.



Analogy
Recipe steps.

6

- **name:** Install Apache
Names the first task.



Analogy
Step 1: Boil Water.

7

apt:
name: apache2
state: present
• **apt** → Module to install packages (Debian/Ubuntu)
• **name** → Package name
• **state: present** → Make sure package is installed.



Analogy
Buy grocery (apache2) and keep it at home. If already there → don't buy again.

8

- **name:** Start Apache Service
Names the second task.



Analogy
Step 2: Switch ON the Fan.

9

service:
name: apache2
state: started
enabled: yes
• **service** → Module to manage services
• **state: started** → Start the service
• **enabled: yes** → Start automatically on reboot.



Analogy
Switch ON the fan and set auto ON when power comes back.

KEY TAKEAWAY

- ★ Ansible Playbook = Recipe for managing servers
- ★ Play decides what to do
- ★ Hosts decide where to do
- ★ Become gives permission
- ★ Tasks are the steps
- ★ Modules are the tools
- ★ YAML indentation keeps everything in order
- ★ Ansible is simple, powerful and repeatable!



Automate
once, run
anywhere!

COMMON ANSIBLE MODULES

Module	Purpose	Real-life Analogy
apt	Install software (Ubuntu/Debian)	Buying groceries
yum	Install software (RHEL/CentOS)	Buying groceries
service	Start/Stop/Manage service	Fan switch
copy	Copy files to remote server	Photocopy / Paste
file	Create/Delete files or directories	Making a cupboard
user	Manage users	School admission
command	Run a command	Giving a direct order
shell	Run shell command	Speaking freely

1. PLAY LEVEL KEYWORDS

Keyword	Description	Example
name	Name of the play	- name: Install Apache
hosts	Target hosts or groups	hosts: webserver
become	Run tasks with sudo/root	become: yes
vars	Define variables for the play	vars: http_port: 80
vars_files	Load variables from files	vars_files: - vars.yml
gather_facts	Collect system facts (Default: yes)	gather_facts: no
tasks	List of tasks to execute	tasks:
roles	Use roles in a play	roles: - common
handlers	Define handlers to run on change	handlers:
pre_tasks	Tasks to run before main tasks	pre_tasks:
post_tasks	Tasks to run after main tasks	post_tasks:
any_errors_fatal	Stop play on any host failure	any_errors_fatal: yes
max_fail_percentage	Abort if failure % exceeds limit	max_fail_percentage: 20
serial	Run play on limited hosts at a time	serial: 2
strategy	Execution strategy (linear/free/debug)	strategy: free
connection	Connection type (ssh/local)	connection: local
remote_user	SSH user	remote_user: ubuntu
environment	Environment variables	environment: http_proxy: http://...



2. TASK LEVEL KEYWORDS

Keyword	Description	Example
name	Name of the task	- name: Install Apache
module	Module to use (apt, copy, service etc.)	apt: name: apache2
args	Arguments for modules	copy: src: file.txt dest: /tmp/file.txt
register	Store task result in variable	register: result
when	Conditional execution	when: ansible_os_family == "Debian"
loop	Loop over items	loop: - item1 - item2
with_items (legacy)	Old way of looping	with_items: - item1 - item2
notify	Notify a handler	notify: Restart Apache
tags	Run task(s) by tags	tags: - install
ignore_errors	Ignore task failure	ignore_errors: yes
changed_when	Define when task is considered changed	changed_when: false
failed_when	Define when task is considered failed	failed_when: result.rc != 0
until	Retry until condition becomes true	until: result is succeeded retries: 5 delay: 10
async	Run task asynchronously	async: 300 poll: 10
become	Override become for task	become: no



3. VARIABLE & DATA KEYWORDS

Keyword	Description	Example
vars	Define variables in play	<pre>vars: port: 8080</pre>
vars_files	Include variable files	<pre>vars_files: - vars.yml</pre>
set_fact	Create variable from task result	<pre>set_fact: myvar: "{{ result.stdout }}"</pre>
register	Register output of a task	<pre>register: cmd_out</pre>
default	Default value if var undefined	<pre>{{ myvar default('NA') }}</pre>
lookup	Lookup external data	<pre>{{ lookup('env', 'HOME') }}</pre>
template	Use Jinja2 template file	<pre>template: src: app.conf.j2 dest: /etc/app.conf.j2</pre>
include_vars	Load variables dynamically	<pre>include_vars: vars.yml</pre>
extra_vars	Pass variables at runtime	<pre>ansible-playbook play.yml -e "key=value"</pre>



4. FLOW CONTROL & CONDITIONAL KEYWORDS

Keyword	Description	Example
when	Run task only when condition true	<pre>when: ansible_distribution == "Ubuntu"</pre>
skip_tags	Skip tasks by tags	<pre>ansible-playbook play.yml --skip-tags test</pre>
only_tags	Run only tasks with tags	<pre>ansible-playbook play.yml --tags install</pre>
until	Repeat until success	<pre>until: result.rc == 0 retries: 5 delay: 5</pre>
block	Group of tasks	<pre>block: - task1 - task2</pre>
rescue	Tasks if block fails	<pre>rescue: - debug: msg="failed"</pre>
always	Always run tasks	<pre>always: - debug: msg="done"</pre>



5. HANDLER KEYWORDS



Keyword	Description	Example
handlers	Define handlers section	handlers:
listen	Group handlers by name	listen: Restart Web
name	Name of the handler	- name: Restart Apache
notify	Notify handler from task	notify: Restart Web

7. IMPORT / INCLUDE KEYWORDS



Keyword	Description	Example
import_playbook	Import another playbook (at parse time)	import_playbook: other.yml
include_playbook	Include another playbook (at run time)	include_playbook: other.yml
import_tasks	Import tasks from file (at parse time)	import_tasks: tasks.yml
include_tasks	Include tasks from file (at run time)	include_tasks: tasks.yml
import_role	Import a role (static)	import_role: name: common
include_role	Include a role (dynamic)	include_role: name: common

6. LOOPING KEYWORDS (MODERN WAY)



Keyword	Description	Example
loop	Loop over a list	loop: [1,2,3]
loop_control	Control loop behaviour	loop_control: loop_var: item
with_dict	Loop over dictionary	loop: "{{ mydict dict2items }}"
with_sequence	Loop over sequence	loop: "{{ range(1,5) list }}"
join	Join list items	{{ mylist join(',') }}
product	Cartesian product	loop: "{{ list1 product(list2) list }}"

8. MISCELLANEOUS USEFUL KEYWORDS



Keyword	Description	Example
debug	Print variable / message	debug: var: myvar
assert	Assert conditions	assert: that: myvar == 1
fail	Force task to fail	fail: msg: "Something went wrong"
meta	Special Ansible actions	meta: end_play
delegate_to	Run task on other host	delegate_to: localhost
run_once	Run task once for all hosts	run_once: yes
delegate_facts	Delegate facts gathering	delegate_facts: yes
no_log	Hide sensitive output	no_log: yes



INFRASTRUCTURE AS CODE (IaC)

Manage Infrastructure with Code, Not Clicks



Infrastructure as Code (IaC) is a modern DevOps practice where servers, networks, and cloud resources are **automatically provisioned and managed using code** instead of manual configuration. It helps maintain **consistency, speed, and reliability** across deployments.

KEY BENEFITS OF IaC



Automates infrastructure setup using scripts or configuration files.



Ensures consistent environments across development, testing, and production.



Enables faster and repeatable deployments.



Reduces human errors caused by manual configuration.



Improves scalability and disaster recovery.



Supports version control for infrastructure changes (like Git).



It treats infrastructure components like software, allowing you to use development practices like version control, testing, and CI/CD pipelines.

IaC IN THE DEVOPS LIFECYCLE



Enabling CI/CD

IaC integrates seamlessly into CI/CD pipelines. A code change can trigger a pipeline that provisions or updates the infrastructure and deploys the application.



Creating Ephemeral Environments

Teams can instantly spin up new, production-like environments for dev, test, or UAT, allowing realistic testing and earlier bug detection.



Breaking Down Silos

Infrastructure code lives in the same VCS as application code. Teams collaborate through pull requests and share a common understanding of the system.

POPULAR IaC TOOLS

1. INFRASTRUCTURE PROVISIONING TOOLS (Create, Modify, Destroy Infrastructure)



Terraform

Open-source by HashiCorp. Cloud-agnostic, supports AWS, Azure, GCP & more. Uses declarative language (HCL) and excels in multi-cloud management.



AWS CloudFormation

Native IaC tool for AWS. Define AWS resources in JSON or YAML templates. Deeply integrated with all AWS services.

2. CONFIGURATION MANAGEMENT TOOLS (Configure Software on Existing Servers)



Ansible

Open-source, simple & agentless. Uses YAML playbooks, operates over SSH. Supports both imperative & declarative.



Puppet

Agent-based, model-driven & declarative. Puppet Master manages agents and enforces the desired state.



Chef

Agent-based, uses Ruby-based DSL. Imperative approach with fine-grained control via recipes & cookbooks.



Salt Stack (Salt)

High-speed, event-driven automation. Agent-based or agentless over SSH. Known for performance & scalability.

3. CONTAINER ORCHESTRATION TOOLS (Manage Containerized Applications)



Kubernetes (K8s)

De-facto standard for container orchestration. Automates deployment, scaling, and management of containerized apps using declarative YAML manifests.

Example (Kubernetes YAML)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: myapp:latest
          ports:
            - containerPort: 80
```

COMMON USE CASES FOR IaC



Cloud Computing

Provision/manage cloud resources like VMs, storage, databases, VPCs.



DevOps & CI/CD

Automate creation of dev, staging, and prod environments in CI/CD pipelines.



Web Application Deployment

Define & deploy entire application stack: web servers, DBs, load balancers.



Networking

Automate networks, subnets, security groups, firewalls, routing, etc.



Big Data

Automate deployment of big data infra like Hadoop, Spark clusters, etc.



Disaster Recovery

Quickly recreate production infra in a different region during failures.

COMMON CHALLENGES & PITFALLS



Learning Curve

IaC tools and concepts can be complex.
Teams need time to learn tools and declarative paradigms.



State Management

State file can become a bottleneck. If corrupted, lost, or locked, it becomes hard to manage infra.



Configuration Drift

Manual changes cause infra to drift from code state, breaking the "single source of truth".



Tooling Complexity

Choosing the right tool (or combination) is difficult.
E.g., Terraform for provisioning + Ansible for config.

THE FUTURE OF IaC: GITOPS & AIOPS



GitOps

Git repository is the single source of truth.
An automated agent (Argo CD, Flux, etc.) continuously monitors Git and the live infra, automatically applying changes to the cluster.
Makes CI/CD more automated, consistent & auditable.



AIOps

AI/ML with IaC enables self-healing infrastructure.
AI detects anomalies, finds root cause, and triggers IaC-defined remediation (e.g., provisioning a new server) without human intervention.

HOW IaC WORKS (HIGH LEVEL FLOW)



Write Code

Define infrastructure using IaC tools (Terraform, Ansible, etc.)



Version Control

Store code in Git repository for tracking & collaboration



CI/CD Pipeline

Code changes trigger pipeline (validate, test, plan/apply)



Provision / Configure

IaC tool provisions or configures infrastructure as per code



Monitor & Maintain

Continuously monitor and maintain infra and repeat.



SUMMARY

IaC brings automation, consistency, and speed to infrastructure management. It is a foundation of modern DevOps and Cloud-Native world.



Automation



Consistency



Speed



Reliability



Collaboration